



Distractor-Aware Memory-Based Visual Object Tracking

Jovana Videnović

Master's study programme Computer and Information Science, study field Data Science

Faculty of Computer and Information Science

Supervisor: prof. dr. Matej Kristan
Cosupervisor: doc. dr. Alan Lukežič

Copyright: This work is licensed under a CC-BY-4.0 license. The software and results produced in this work are released under GPL-3.0-or-later at <https://github.com/jovnavidenovic/DAM4SAM>.

Recent emergence of memory-based video segmentation methods such as SAM2 has led to models with excellent performance in segmentation tasks, achieving leading results on numerous benchmarks. However, these models are not fully adjusted for visual object tracking, where distractors (i.e., objects visually similar to the target) pose a key challenge. In this paper we propose a distractor-aware drop-in memory module and introspection-based management method for SAM2, leading to DAM4SAM. Our design effectively reduces the tracking drift toward distractors and improves redetection capability after object occlusion. To facilitate the analysis of tracking in the presence of distractors, we construct DiDi, a Distractor-Distilled dataset. DAM4SAM outperforms SAM2.1 on thirteen benchmarks and sets new state-of-the-art results on ten. Furthermore, integrating the proposed distractor-aware memory into a real-time tracker EfficientTAM leads to 11% improvement and matches tracking quality of the non-real-time SAM2.1-L on multiple tracking and segmentation benchmarks, while integration with edge-based tracker EdgeTAM delivers 4% performance boost, demonstrating a very good generalization across architectures.

Visual object tracking | Tracking and segmentation | Memory-based models

General visual object tracking is a core task in computer vision that involves localizing an arbitrary target across a video, by a bounding box or a segmentation mask, given a single example in the first frame. Over the past decade, the significant progress has been driven by deep learning and transformer architectures [1–3], the availability of diverse training datasets [4–6], and the development of challenging benchmarks [7–10]. Modern trackers are now widely used in applications such as surveillance, video editing, and sports analytics.

Nevertheless, a substantial source of tracking failures remain nearby objects that visually resemble the tracked object. These regions, called distractors, increase the localization uncertainty and mislead the tracker (Figure 1). Distractors can be classified as external (nearby objects similar to the target) or internal (self-similar patterns within the target itself) when only a part is being tracked. While internal distractors mainly reduce localization accuracy, external distractors present a particular challenge when the target temporarily leaves and re-enters the frame, frequently causing irreversible tracking drift.

Various approaches have been proposed to reduce the visual ambiguity caused by distractors. These include learning discriminative features [1, 11–14] or explicitly modeling the foreground-background by dedicated modules [15–18]. An emerging paradigm, already positioned at the top of the major benchmarks [8, 9, 19], are memory-based frameworks, which localize the target by pixel association with the past tracked frames [2, 20, 21].

Memory-based methods construct the target model by storing multiple past images with the target segmented by the tracker, thereby implicitly encoding the potential distractors segmented as the background. While the tendency of initial memory-based models was to continually add arriving frames to the memory, [21] argued that the visual redundancy in large memory leads to reduced localization capability due to the nature of cross-attention. They show that limiting the memory to the most recent frames and temporally time-stamping them in fact improves tracking. This paradigm was further validated by the recent video segmentation foundation model SAM2 [22], which sets a solid state-of-the-art across several video segmentation and tracking benchmarks [9, 23–26].

We argue that while the recent target appearances in the memory are required for accurate segmentation, another type of memory is required to distinguish the target from challenging distractors. To support this claim, we propose a new distractor-aware memory (DAM) and a memory maintenance mechanism for SAM2-like architectures. The new memory is divided by its tracking functionality into two parts: the recent appearances memory (RAM) and distractor-resolving memory (DRM). While RAM contains the recent target appearances sampled at regular intervals, DRM contains anchor frames that help discriminate the target from critical distractors. A novel DRM updating mechanism is proposed that exploits the multiple hypothesis output of SAM2 (see Figure 1), which has been previously overlooked in the tracking research.

In addition, we observe that standard benchmarks contain many sequences, which are no longer considered challenging by modern standards. The high performance on these sequences overwhelms the total score, saturates the benchmarks, and does not properly

Vizualno sledenje objektom na podlagi pomnilnika z upoštevanjem distraktorjev

Nedavni pojav metod za segmentacijo videa, ki temeljijo na pomnilniku, kot je SAM2, je privedel do modelov z odlično zmogljivostjo pri segmentacijskih nalogah, saj dosegajo vodilne rezultate na številnih podatkovnih zbirkah. Vendar ti modeli še niso v celoti prilagojeni vizualnemu sledenju objektov, kjer distraktorji (tj. objekti, ki so vizualno podobni sledenemu objektu) predstavljajo ključen izziv. V tem članku predlagamo zamenljiv pomnilniški modul, ki upošteva moteče objekte, ter metodo upravljanja, temelječo na introspekciji, za SAM2, kar vodi do metode DAM4SAM. Naša zasnova učinkovito zmanjša odnašanje sledenja proti distraktorjem in izboljša zmožnost ponovnega zaznavanja po zakritju objekta. Za lažjo analizo sledenja v prisotnosti motečih objektov smo pripravili DiDi, podatkovno zbirko, destilirano glede na moteče objekte. DAM4SAM prekaša SAM2.1 na trinajstih podatkovnih zbirkah in postavlja nove najboljše rezultate na desetih. Poleg tega vključitev predlaganega pomnilnika, ki upošteva moteče objekte, v sledilnik EfficientTAM v realnem času prinese 11% izboljšanje in doseže kakovost sledenja, primerljivo z nerealnočasovnim modelom SAM2.1-L na več primerjalnih testih sledenja in segmentacije, medtem ko vključitev v robni sledilnik EdgeTAM prinese 4% izboljšanje zmogljivosti, kar kaže na zelo dobro posplošljivost med različnimi arhitekturami.

Vizualno sledenje objektov | Sledenje in segmentacija | Pomnilniški modeli

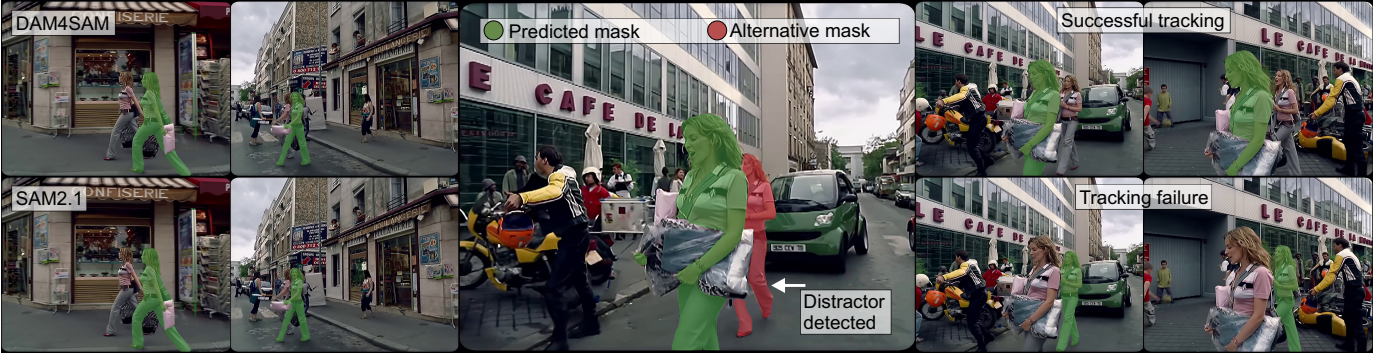


Figure 1. Comparison of DAM4SAM (top row) with SAM2.1 (bottom row). The proposed distractor-aware memory (DAM), integrated into SAM2.1 (referred to as DAM4SAM), resolves visual ambiguities and enhances tracking robustness. The DAM update is triggered by divergence between the predicted and alternative masks in SAM2.1 multi-mask output (shown in the middle column). At this point, both SAM2.1 and DAM4SAM still track the target correctly, but one of the alternative mask segments the distractor, which triggers the DAM update, leading to improved performance compared to SAM2.1 (last two columns).

expose the tracking advances. To address this, we semi-automatically distill several benchmarks into a distractor-distilled tracking dataset (DiDi). The dataset contains 180 sequences, each annotated with the initial segmentation mask. In our experimental analysis, we show that DiDi clearly exposes the tracking improvements of trackers with explicit distractor handling compared to others, which is not the case for other common benchmarks [4, 5, 10].

In summary, we make the following contributions:

- We propose a drop-in distractor-aware memory (DAM) framework for memory-based trackers. To the best of our knowledge, this is the first memory formulation that divides and updates the memory with respect to its function in tracking.
- We integrate DAM into SAM2.1 [22] model, leading to DAM4SAM. Without requiring additional training, DAM4SAM improves a baseline tracker by notable margins on thirteen visual object tracking and video object segmentation benchmarks, setting state-of-the-art results on ten of them, including LaSOT_{ext} [10] and LVOSv2 [27].
- We integrate DAM into the real-time SAM2-based model, named Efficient Track Anything (TAM) [28]. This demonstrates DAM’s generalization capability, as it boosts TAM’s performance by up to 11%. Notably, DAM-enhanced TAM achieves tracking accuracy comparable to SAM2.1-L on several datasets, despite using $6.5\times$ fewer parameters and being $2\times$ faster.
- We construct a new DiDi dataset. The dataset consists of 180 sequences, with average length of 1,500 frames, making it the largest and most challenging distractor-oriented tracking dataset.

A preliminary version of this work was published in [29]. This paper extends it in several directions: we extend the related work by charting historical developments in tracking that led to memory-based trackers and include concurrent SAM2 extensions, we expand the experimental evaluation to include six additional datasets, including three video object segmentation benchmarks, extend the ablation study and present the sensitivity analysis, demonstrate generalization capabilities of DAM, and provide detailed qualitative insights with a failure case analysis.

1. Related work

Early deep learning methods in visual object tracking focused on fine-tuning pre-trained networks, or their parts, during inference [30, 31]. While these approaches outperformed traditional methods such as correlation filter-based trackers [32, 33], they were computationally intensive and struggled with robustness to object appearance variations. Fine-tuning allowed the model to adapt to the specific target object, but the need for online updates introduced inference latency.

A major shift occurred with the emergence of CNN-based Siamese trackers [11, 34–36], which eliminated online training by adopting a template-matching paradigm using a reference template from the first frame. These methods brought notable efficiency gains, but struggled with distinguishing the target from semantically similar backgrounds. Their reliance on local search strategies also limited robustness under full occlusion or out-of-view scenarios.

The issues of CNN-based Siamese trackers were partially addressed by attention-based models [1, 37], which extended the Siamese architecture by employing attention mechanisms to enable fusing template and search region features. For instance, they achieved state-of-the-art performance without even updating the template [1], while combining efficiency and architectural simplicity. The next generation of transformer-based trackers incorporated information beyond the initial frame for localization of the target [12, 14]. This family of methods leveraged transformer-based architectures to dynamically update the target template using predictions from the previous frame, which enabled adaptive and temporally consistent tracking, improving robustness to occlusions, deformations, and appearance changes.

In recent years, the tracking paradigm of top-performers has shifted towards video segmentation-based memory networks [2, 17, 20–22]. These methods embed predictions from past frames into memory, therefore extending contextual information beyond just the initial or the previous frame. The attention mechanism is typically used to link frame representations stored in the memory with features extracted from the current frame. In initial methods such as by [2], arriving frames were continually added to the memory. This led to theoretically unbounded increase in computational complexity and GPU memory. The issue was addressed in [17, 20] by using multiple memory storages and efficient compression schemes to capture different temporal contexts, enhancing performance on long-term videos. Later, [21] proposed to restrict the memory to the most recent frames with temporal stamping, which led to improved localization.

Recently, [22] introduced a restricted memory-based video segmentation foundation model SAM2, which demonstrated remarkable performance across a number of video object segmentation and tracking benchmarks. The model retains the initial frame and the six most recent frames in the memory. Its remarkable performance spawned a number of follow-up works that built on the proposed architecture. [28] proposed EfficientTAM, a lightweight tracking model with results comparable to SAM2, higher speed and smaller model sizes. Building on SAM2 [22], several concurrent works have led to extensions tackling specialized tasks, including long-term video object segmentation [38], motion-aware object tracking [39], and referring segmentation in videos [40]. SAM2Long [38] selects the optimal segmentation trajectory via a constrained tree search, thereby improving performance on long sequences. SAMURAI [39] integrates motion cues into memory selection and mask refinement, which enables remarkable handling of crowded scenes with fast motions. These studies are concurrent and closely related to ours, as they share the core idea of adapting memory to address specific challenges. However, DAM4SAM differs by introducing a new memory model that separates the memory according to its function and updates it by exploiting SAM2 output ambiguity for explicit distractor detection, resulting in improved segmentation accuracy and robustness.

Several works have recognized the challenge of tracking in presence of distractors and proposed different solutions to the problem. [41] cast the problem as a multi-target tracking setup and propose KeepTrack. The model identifies target candidates and potential distractors, which are then associated with previously propagated identities using a learned association network. However, the method relies on accurate detection, and cannot address internal distractors in practice. [18] showed that treating target localization accuracy and robustness as two distinct tasks is beneficial, particularly in scenarios involving distractors. Nevertheless, these distractor handling approaches lead to complex pipelines, potentially combining several methods with many hand-crafted rules, thus preventing them from fully exploiting the learning potential of modern frameworks. Alternatively, memory-based methods [17, 20–22] have the capacity to implicitly handle distractors, since they store entire images and apply a learnable localization by segmentation. We make a step further in DAM4SAM by splitting the memory according to its function of distractor learning and frame-to-frame segmentation, and develop a novel memory update technique based on distractor detection.

2. DAM4SAM

This section describes the new distractor-aware memory (DAM) model for SAM2, which is a drop-in module for SAM2 and does not require additional training. For completeness, SAM2 architecture is described in Section 2.1, while the new model is described in Section 2.2.

2.1. SAM2 preliminaries. SAM2 extends the Segment Anything Model (SAM) [42], originally developed for interactive class-agnostic image segmentation, to video segmentation. It consists of four main components: (i) image encoder, (ii) prompt encoder, (iii) memory bank, and (iv) mask decoder.

Image encoder applies the Hiera¹ backbone [43] to embed the input image. Interactive

¹Hiera-L version is used in all our experiments.

inputs (e.g., positive/negative clicks) are absorbed by the prompt encoder and used for output mask refinement. Note that in the non-interactive general object tracking setup considered in this paper, these prompts are used only during the initialization step and are not applied during tracking. For all subsequent frames, the prompt inputs are set to empty points with zero-value embeddings. The memory bank consists of the encoded initialization frame with a user-provided segmentation mask and six recent frames with segmentation masks generated by the tracker. Temporal encodings are applied to the six recent frames to encode the frame order, while such encoding is not applied to the initialization frame to indicate its unique property of being a single supervised training example, and thus serves as a sort of target prior model.

The memory bank transfers pixel-wise labels onto the current image by cross-attending the features in the current frame to all memory frames, producing memory-conditioned features. The features are then decoded by the mask decoder, which predicts three output masks along with the prediction of the IoU with the (unknown) ground truth for each. The mask with the highest predicted IoU score is chosen as the tracking output.

SAM2 applies a variant of the memory management proposed in [21], where the initialization frame is always kept in the memory, while the six recent frames are updated at every new frame by a first-in-first-out protocol. The memory and the management mechanism are visualized in Figure 2, while the reader is referred to [22] for more details.

SAM2.1 version further improves upon SAM2 by enhancing the handling of small and visually similar objects through additional training augmentations, and by improving robustness to occlusions via training on longer video sequences. Unless stated otherwise, SAM2.1-L model checkpoint is adopted as the baseline in our experiments.

2.2. Distractor-aware memory. Related works [2, 20–22] have clearly demonstrated the importance of the most recent frames to address target appearance changes and for ensuring accurate segmentation. However, a different type of frames is required to prevent drifting in the presence of critical distractors and for reliable target re-detection.

We propose to separate the memory with respect to its function during tracking into (i) *recent appearance memory* (RAM) and (ii) *distractor resolving memory* (DRM). RAM and DRM together form the distractor-aware memory (DAM), visualized in Figure 2. The function of RAM is to ensure segmentation accuracy in the considered frame. Thus we design it akin to the current SAM2 [22] memory, as a FIFO buffer with $\frac{1}{2}N_{\text{DAM}} = 3$ slots, containing the most recent target appearances with temporal encoding to identify temporally more relevant frames for the task.

On the other hand, DRM serves for ensuring tracking robustness and re-detection. It should contain accurately segmented frames with critical recent distractors, including the initialization frame. These segmented frames consist of labelled pixels considered either as *belonging to the target* or as *not belonging to the target*. The latter includes *distractor* regions and keeping such frames in DRM allows leveraging SAM2’s internal mechanism to resolve confusion between the target and the potential distractor in the current frame. The DRM is composed of a slot reserved for the initialization frame and a FIFO buffer with $\frac{1}{2}N_{\text{DAM}} = 3$ anchor frames updated during tracking². Since the purpose of DRM is to encode critical information for resolving distractors, it does not apply temporal encoding. Note that the pre-trained SAM2 already contains the building blocks to implement the proposed memory structure.

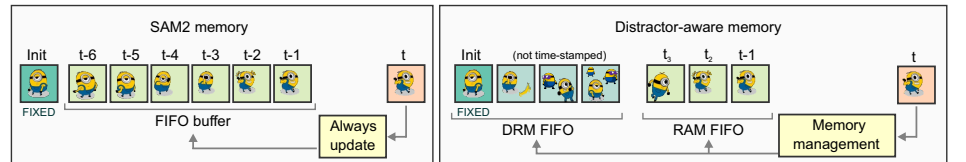


Figure 2. Overview of SAM2 memory and the proposed Distractor-Aware Memory (DAM), which splits the model into *Recent Appearance Memory* (RAM) and *Distractor Resolving Memory* (DRM), and updates them using a new memory management protocol. Note that t_2 and t_3 represent the time steps at which RAM was updated, i.e., every Δ frames. Additionally, RAM and DRM update is prevented on frames where the predicted mask is empty.

2.2.1. RAM management protocol. A crucial element of the memory-based methods is the memory management protocol. To efficiently exploit the available memory slots, the memory should not be updated at every frame, since the consecutive frames are highly correlated. Indeed, [21] argue that visual redundancy in memory should be avoided in attention-based localization. When the memory contains many images with redundant and noisy features, the attention becomes diluted, which can lead to lower weights on the truly relevant features. In visual object tracking, the videos are typically recorded at 30 FPS, resulting in high appearance redundancy between consecutive frames. RAM

²At tracker initialization, RAM occupies all N_{DAM} slots and drops to $\frac{1}{2}N_{\text{DAM}}$ as DRM entries arrive, to fully exploit the available capacity.

is thus updated every $\Delta = 5$ frames, and includes the most recent frame since it is the most relevant for accurate target segmentation in the considered frame.

SAM2 [22] updates the memory at every frame, including when the target is absent. The target is considered absent when the segmentation mask is all zeros. However, even for a very short occlusion, the memory will quickly fill up with frames without the target, which reduces the target appearance diversity in the model, leading to a reduced segmentation accuracy upon target re-appearance. Furthermore, failing to re-detect the target leads to incorrectly updating the memory by an empty mask, which may cause error accumulation and ultimately re-detection failure. We thus propose to not update RAM when the target is not present, i.e., when the predicted target mask is empty.

2.2.2. DRM management protocol. DRM inherits the initial update rules from RAM, i.e., update only when the target is present and at least $\Delta = 5$ frames have passed since the last update. It considers an additional rule to identify anchor frames containing critical distractors. In particular, drifting to a distractor may be avoided by including a past temporally nearby frame with this distractor accurately segmented as the background. As opposed to SAM2, which constantly updates the memory, such frames would quickly be removed, which leads to eventual drift to a distractor and tracking failure.

Recall that SAM2 predicts three output masks and selects the one with the highest predicted IoU (Section 2.1), which means that we can consider it as a multi-hypothesis prediction model. Our preliminary study showed that in the frames before the failure occurs, SAM2 in fact detects such distractors in the alternative two predicted output masks (see Figure 1).

We thus propose a simple anchor frame detection mechanism based on determining hypothesis divergence between the output and alternative masks. A bounding box is fitted to the output mask and to the union of the output mask and the largest connected component in the alternative mask. If the ratio between the area of the two bounding boxes drops below $\theta_{\text{anc}} = 0.7$, the current frame is considered as a potential candidate to update DRM. The anchor frame detection is visualized Figure 3. Note that updating with a grossly incorrectly segmented target would lead to memory corruption and eventual tracking failure. We thus trigger the DRM update only during sufficiently stable tracking periods, i.e., if the predicted IoU score from SAM2 exceeds a threshold $\theta_{\text{IoU}} = 0.8$, and if the mask area is within $\theta_{\text{area}} = 0.2$ of the median area in the last $N_M = 10$ frames. Note that DAM4SAM is mostly insensitive to the exact value of these parameters, as verified in Section 4.1.2.

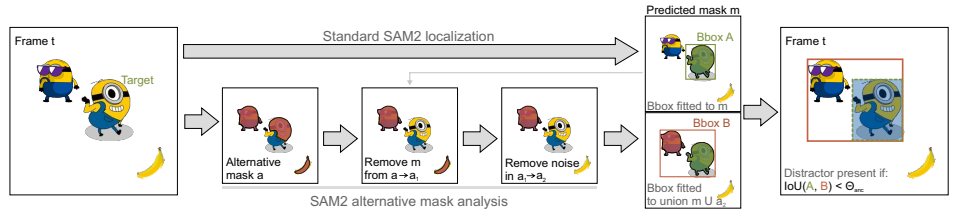


Figure 3. Anchor frame is identified by detecting a distractor through analysis of agreement between SAM2 predicted object mask with the predicted alternative masks.

3. A distractor-distilled dataset

While benchmarks played a major role in the recent visual object tracking breakthroughs, we note that many of these contain sequences, which are no longer considered challenging by modern standards. In fact, most of the modern trackers obtain high performance on these sequences, which overwhelms the total score and under-represents the improvements on challenging situations. To facilitate the tracking performance analysis of the designs proposed in this paper, we semi-automatically distill several benchmarks into a distractor-distilled tracking dataset (DiDi).

We considered validation and test sequences of the major tracking benchmarks, which are known for high-quality annotation, i.e., GoT-10k [5], LaSOT [4], UTB180 [44], VOT-ST2020 and VOT-LT2020 [45], and VOT-ST2022 and VOT-LT2022 [8]. This gave us a pool of 808 sequences. A sequence was selected for the DiDi dataset if at least one-third of the frames passed the distractor presence criterion described next.

A frame was classified as containing non-negligible distractors if it contained a large enough region, visually similar to the target. To make this criterion as much independent as possible from the modern tracking localization methods, we considered the following approach. First, dense image feature maps were extracted from the input image by a DINOv2 [46] backbone. For each pixel in this feature space, we computed its cosine similarity to multiple target feature vectors extracted from the ground truth target region.

The distractor score at each pixel was then calculated as the average cosine similarity between that pixel’s feature vector and the set of target features, resulting in a dense similarity response map. We then thresholded the map to obtain a binary map containing regions similar to the target. To assess the presence of distractors, we computed the ratio of the number of high-similarity pixels outside the target region to those inside. If this ratio exceeded 0.5, the frame was classified as containing distractors.

Table 1. Summary of datasets used for constructing the proposed DiDi dataset.

Dataset	Num. of seqs.
LaSoT [4]	86
UTB-180 [44]	56
VOT2022-ST [8]	20
VOT2022-LT [8]	7
VOT2020-LT [45]	6
GoT10k [5]	4
VOT2020-ST [45]	1
Total	180

Using the aforementioned distractor presence detection, we finally obtained 180 sequences with an average sequence length of 1,500 frames (274,882 frames in total). Each sequence contains a single target annotated by an axis-aligned bounding box. In addition, we manually segmented the initial frames to allow the initialization of segmentation-based trackers. The number of sequences obtained from each source dataset is given in Table 1. Figure 4 shows frames from the proposed DiDi dataset.

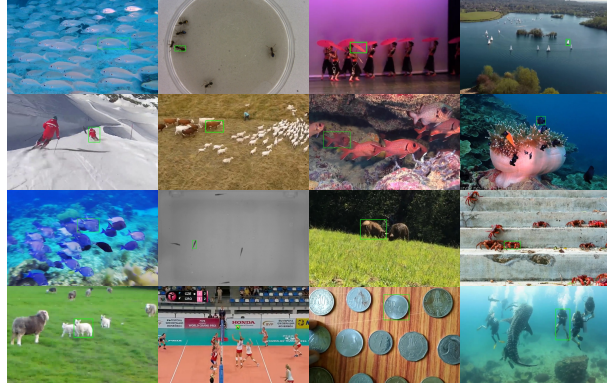


Figure 4. Example frames from the DiDi dataset showing challenging distractors. Targets are denoted by green bounding boxes.

4. Experiments

Various aspects of the proposed DAM for SAM2 are analyzed in the following. Section 4.1 reports an ablation study – it includes a series of experiments to justify the design choices and to demonstrate sensitivity to threshold values. Section 4.2 compares SAM2.1 extension with DAM with the state-of-the-art on the DiDi dataset. Detailed analysis on the challenging VOT tracking-by-segmentation benchmarks is reported in Section 4.3. Comparisons on the standard bounding-box tracking and video object segmentation benchmarks are reported in Section 4.4 and Section 4.5, respectively. In the final two Section 4.6 and Section 4.7, we present qualitative analysis of the proposed tracker and an overview of its failure cases.

4.1. Ablation study.

4.1.1. Architecture design decisions. The design choices of the proposed distractor-aware memory and the management protocol from Section 2.2 are validated on the DiDi dataset from Section 3. We apply the standard VOTS [19] performance evaluation protocol and its metrics since they simultaneously account for short-term as well as long-term tracking performance. Performance is summarized by the tracking quality Q score and two auxiliary measures: robustness (i.e., portion of successfully tracked frames) and accuracy (i.e., the average IoU between the prediction and ground truth during successful tracking). Results are shown in Table 2 and in the AR plot in Figure 5.

We first verify the argument that updating with frames without a target present causes memory degradation. We thus extend SAM2.1, with updating only when the predicted

mask is not empty (denoted as SAM2.1_{PRES}). SAM2.1_{PRES} increases the tracking quality Q by 2.5%, primarily by improved robustness, which justifies our claim.

We next verify the assumption (also claimed in [21]) that frequent updates reduce tracking robustness due to highly correlated information stored in memory. We reduce the update rate in SAM2.1_{PRES} to every 5th frame (SAM2.1 _{$\Delta=5$}). This negligibly improves Q, but does increase the robustness by 1.2%, which supports the claim. We did not observe further performance improvement with increasing Δ .

Finally, we focus on our proposed distractor-resolving memory (DRM), which accounts for the tracking robustness in the presence of distractors. Recall that DRM is updated when a distractor is detected, and under the condition that tracking is reliable – we first test the influence of these two conditions independently. We thus extend SAM2.1 _{$\Delta=5$} with the new DAM and update the DRM part only during reliable tracking periods (SAM2.1_{DRM1}). The tracking accuracy improves slightly, while the robustness increases by 2%. Alternatively, we change the rule to update only when the distractor is detected (SAM2.1_{DRM2}). While robustness remains unchanged with respect to SAM2.1 _{$\Delta=5$} , the accuracy in fact drops. This is expected since distractor detection may be triggered also by the error in the target segmentation, which becomes amplified by the update. To verify this, we next apply both our proposed update DRM rules, arriving to the proposed DAM with SAM2.1 (DAM4SAM for short). Compared to SAM2.1 _{$\Delta=5$} , we observe substantial improvement in tracking quality Q (4%), primarily due to 3.3% boost in robustness as well as 1.3% boost in accuracy, taking the top-right position in the AR plot (Figure 5) among all variants. This conclusively verifies that DRM should be updated with distractor detected only if tracking is sufficiently reliable.

A central element of the distractor detection algorithm from Section 2.2.2 is the connected components operation, whose task is to remove the noise in the predicted mask (e.g., small spurious patches of segmented pixels away from the main target), thus stabilizing distractor detection. To verify the noise removal hypothesis, we replace it by a median filter with kernel size 3×3 (named DAM4SAM_{MED}). The results in Table 2 indicate that the tracking performance is comparable to when using the connected components. The performance drop is negligible, which strongly supports our hypothesis.

Table 2. DAM4SAM architecture justification on the proposed DiDi dataset.

	Quality	Accuracy	Robustness
SAM2.1 (ICLR25)	0.649	0.720	0.887
SAM2.1 _{PRES}	0.665	0.723	0.903
SAM2.1 _{$\Delta=5$}	0.667	0.718	0.914
SAM2.1 _{DRM1}	0.672	0.710	0.932 ③
SAM2.1 _{DRM2}	0.644	0.691	0.913
DAM4SAM _{MED}	0.692 ②	0.726 ②	0.941 ②
DAM4SAM	0.694 ①	0.727 ①	0.944 ①
DRM _{tenc}	0.669	0.711	0.925
RAM _{last}	0.685 ③	0.724 ③	0.932 ③

In Section 2.2 we claim that DRM part of the memory should not be time-stamped, since the frame influence on the distractor resolving in the current frame should not be biased by the temporal proximity and should serve as a time-less prior. To test this claim, we modify DAM4SAM by using temporal encodings in DRM (except for the initialization frame) – we denote it as DRM_{tenc}. The tracking quality drops by 3.6%, which confirms the claim.

We further inspect the updating regime in RAM, which always includes the most recent frame, but updates the memory slots every 5th frame. DAM4SAM is modified to update all RAM slots at every 5th frame (RAM_{last}). This results in a slight tracking quality decrease (1.3%), which indicates that including the most recent frame in RAM is indeed beneficial, but not critical.

4.1.2. Sensitivity analysis. We analyze the sensitivity of the proposed DAM4SAM to the exact value of the manually determined parameters, defined in Section 2.2. Experiments were conducted on the VOT2022 [8] dataset using the unsupervised experiment to ensure fast execution and compute the average overlap (AO) as the performance measure.

Firstly, we examined the impact of different memory splits, specifically 2 frames in DRM and 4 frames in RAM, and vice versa. The results showed no significant changes, with performance drops of up to 0.3%, indicating robustness to memory allocation variations.

We then evaluate the impact of varying the memory size in both RAM and DRM modules. Specifically, we experiment with configurations using fewer and more memory

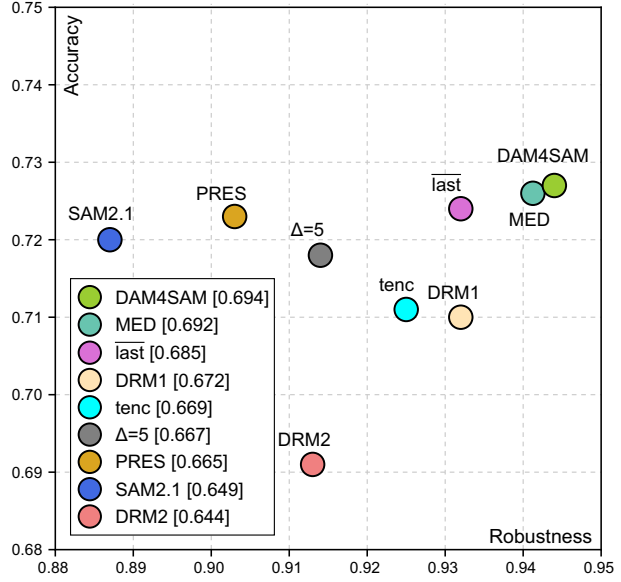


Figure 5. Accuracy-robustness plot on DiDi for the ablated versions of DAM4SAM. The tracking quality is given at each label. In interest of clarity, shortened labels correspond to the subscript components of the full model names in Table 2 are used.

slots than the default. The tested settings include: 2 slots in both RAM and DRM, 4 slots in both RAM and DRM, and an asymmetric configuration with 6 slots in RAM and 3 slots in DRM. The results, measured by Average Overlap (AO) on the VOT2022-ST benchmark, are summarized in Table 3. The default configuration with three slots in both RAM and DRM yields the best overall performance. A marginally lower performance is observed when reducing the memory size to two slots in each memory module. In contrast, increasing the memory capacity beyond this configuration leads to degraded results. As noted in [21], expansion of memory bank often leads to redundant information in the memory, which adversely affects attention weighting in memory attention operations. Consequently, memory expansion leads to decreased overall tracking and segmentation performance. These findings are consistent with our observations when RAM=4 & DRM=4 and RAM=6 & DRM=3.

Table 3. DAM4SAM memory size ablation on VOT2022 dataset.

# RAM slots	# DRM slots	AO (%)
2	2	78.1
3	3	79.2
4	4	74.9
6	3	75.9

Next, we analyzed the influence of the threshold for the ratio between the target and alternative predicted masks, i.e. $\Theta_{anc} = 0.7$. This ratio is used for preemptive update upon distractor detection and is thus crucial for our distractor-aware memory. As shown in Figure 6, tracking performance remains highly stable for a wide range of $\Theta_{anc} \in [0.6, 0.9]$ demonstrating the robust design of the tracker.

Similarly, we evaluated the IoU score threshold ($\Theta_{IoU} = 0.8$), which determines the reliability of the predicted mask for distractor testing. Results in Figure 6 show that tracking performance remains consistent for a wide range of parameters ($\Theta_{IoU} \in [0.5, 0.8]$), scoring almost identical AO.

The mask area threshold was also tested, i.e. $\Theta_{area} = 0.2$. This threshold is used to determine the tracking stability and is together with the Θ_{IoU} a necessary condition to trigger the DRM update. Tracking performance remained stable across variations of this parameter.

Finally, we evaluated the impact of the window size used in the median calculation for temporally consistent predictions, starting with an initial value of $N_M = 10$ and exploring a range of values three times larger and smaller. The performance variations were up to 1.5% AO, indicating stability.

Sensitivity analysis for parameters Θ_{anc} , Θ_{IoU} , Θ_{area} and N_M across a wide range of thresholds is summarized in Figure 6. DAM4SAM demonstrates stable tracking

performance, confirming that it is not sensitive to the exact parameter values.

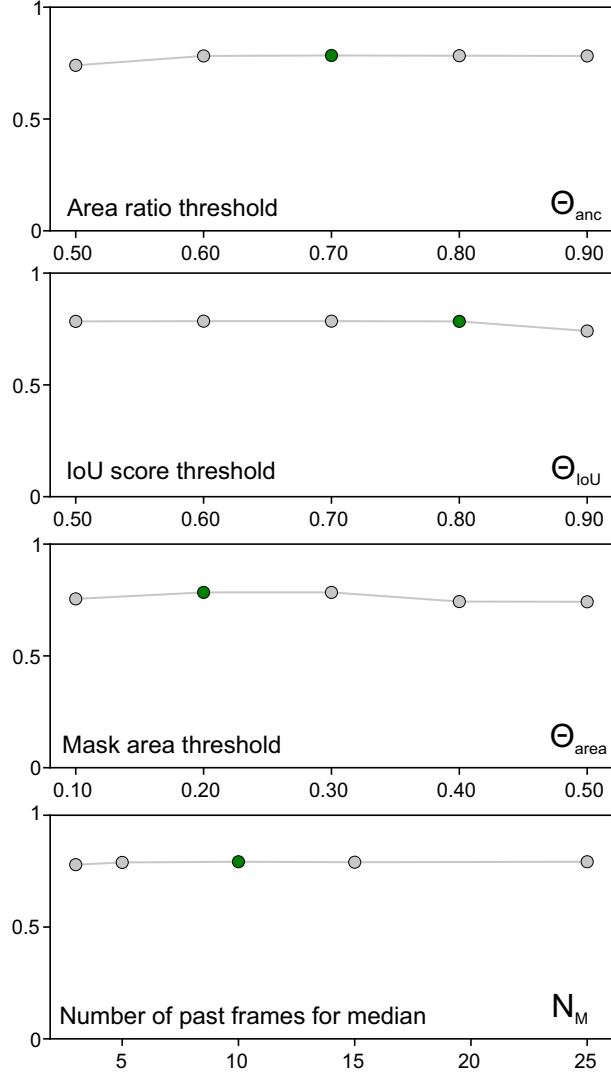


Figure 6. Sensitivity of the DAM4SAM to different values of parameters: Θ_{anc} , Θ_{IoU} and Θ_{area} and N_M . Experiments were done on VOT2022 using average overlap as the performance measure. The selected parameter value is marked by a green circle.

4.2. Comparison on DiDi. DAM4SAM is evaluated on the DiDi dataset against the baseline SAM2.1 [22], as well as several recent state-of-the-art trackers, including TransT [1], SeqTrack [14], AQATrack [47], and AOT [2]. We also compare with trackers that incorporate explicit distractor handling mechanisms, such as KeepTrack [41], Cutie [17], and ODTrack [16]. Additionally, we include the latest, yet unpublished, SAM2 extensions: SAM2Long [38], which improves long-term tracking via memory tree, and SAMURAI [39], which introduces motion-aware memory.

Results in Table 4 reveal the advantages of the trackers with an explicit distractor-handling mechanism over the other trackers. Consider two similarly complex recent trackers SeqTrack and ODTrack, which are both based on the ViT-L backbone. On classical benchmarks like LaSoT, LaSoT_{ext} and GoT10k, ODTrack outperforms SeqTrack by 2%, 6% and 4%, respectively (see Table 9). However, the performance gap increases to 15% on DiDi (Table 4), which confirms that distractors are indeed a major challenge for modern trackers, and that DiDi has a unique ability to emphasize the tracking capability under these conditions and to expose tracking design weaknesses.

Focusing on the evaluation of the proposed tracker, DAM4SAM outperforms all trackers – the standard state-of-the-art trackers as well as trackers with explicit distractor handling mechanism. In particular, DAM4SAM outperforms state-of-the-art distractor-aware trackers ODTrack and Cutie, by 14% and 21%, respectively.

We compare the proposed DAM4SAM with the concurrent work SAMURAI [39]. Results show that DAM4SAM outperforms SAMURAI in tracking quality by 2% on DiDi, mostly due to the higher robustness (i.e., DAM4SAM tracks longer than SAMURAI). This result demonstrates the superiority of our new DAM memory and the management

protocol for distractor handling, which is also less complex than the concurrent counterpart proposed in SAMURAI.

Compared to another unpublished tracker with the alternative memory design SAM2.1Long [38], DAM4SAM outperforms it by a 7% tracking quality boost, indicating superiority of our proposed memory. The results reveal that the major source of the performance boost is the DAM4SAM tracking robustness, which means it fails less often and thus much better handles distractors. In fact, a close inspection of the results shows that SAM2.1Long performs on par with the baseline SAM2.1, which indicates that the long-term memory update mechanism of SAM2.1Long does not improve performance in the presence of distractors. Finally, comparing DAM4SAM to the baseline SAM2.1 reveals 7% boost in tracking quality, again, attributed mostly due to the improved robustness (6%). These results validate the benefits of the proposed DAM and its management protocol in handling challenging distractors.

Table 4. State-of-the-art comparison on DiDi dataset. DAM4SAM, SAMURAI, and SAM2.1Long use SAM2.1-L checkpoint, and its results are also included in the table for reference.

	Quality	Acc.	Rob.
SAMURAI (arXiv24)	0.680 ②	0.722 ③	0.930 ②
SAM2.1Long (arXiv24)	0.646	0.719	0.883
ODTrack (AAAI24)	0.608	0.740 ①	0.809
Cutie (CVPR24)	0.575	0.704	0.776
AOT (NeurIPS21)	0.541	0.622	0.852
AQATrack (CVPR24)	0.535	0.693	0.753
SeqTrack (CVPR23)	0.529	0.714	0.718
KeepTrack (ICCV21)	0.502	0.646	0.748
TransT (CVPR21)	0.465	0.669	0.678
SAM2.1 (ICLR25)	0.649 ③	0.720	0.887 ③
DAM4SAM	0.694 ①	0.727 ②	0.944 ①

4.3. Comparison on VOT challenges. The VOT initiative [48, 49] is the major tracking initiative, providing challenging datasets for their yearly challenges. In contrast to most of the tracking benchmarks, the targets are annotated by segmentation masks, which allows more accurate evaluation of segmentation trackers, compared to the classic bounding-box benchmarks. We consider two recent single-target challenges VOT2020 [45] and VOT2022 [8], and the most recent multi-target challenge VOTS2024 [9]. We further evaluate the proposed DAM4SAM under real-time tracking constraints on VOT2022-RT [8] challenge³.

VOT2020 benchmark [45] consists of 60 challenging sequences, while trackers are run using anchor-based protocol [45] to maximally utilize each sequence. Tracking performance is measured by the accuracy and robustness, summarized by the primary measure called the expected average overlap (EAO).

Results on VOT2020 are shown in Table 5. The proposed DAM4SAM outperforms all compared trackers. In particular, it outperforms the recently published MixViT [50] by 25% EAO, improving both, accuracy and robustness. DAM4SAM outperforms also the VOT2020 challenge winner RPT [51] by a large margin (37.5% in EAO). Comparing DAM4SAM to the original SAM2.1, the EAO is boosted by 7%, while accuracy and robustness are improved by 2.7% and 2.1%, respectively.

Table 5. State-of-the-art comparison on the VOT2020 benchmark. The challenge winner is marked by ②.

	EAO	Acc.	Rob.
ODTrack (AAAI24)	0.605 ③	0.761	0.902 ③
MixViT (TPAMI24)	0.584	0.755	0.890
SeqTrack (CVPR23)	0.561	-	-
MixFormer (CVPR22)	0.555	0.762 ③	0.855
RPT ② (ECCVW20)	0.530	0.700	0.869
OceanPlus (ECCV20)	0.491	0.685	0.842
AlphaRef (CVPR21)	0.482	0.754	0.777
AFOD (ECCV20)	0.472	0.713	0.795
SAM2.1 (ICLR25)	0.681 ②	0.778 ②	0.941 ②
DAM4SAM	0.729 ①	0.799 ①	0.961 ①

³SAM2.1 and DAM4SAM were evaluated on the machine with the AMD EPYC 7763 64-Core 2.45 GHz CPU and Nvidia A100 40GB GPU.

VOT2022 benchmark [8] uses a refreshed dataset with 62 sequences (simplest sequences removed, and more challenging added). Table 6 includes the challenge top-performers, including the winner MS_AOT [2], as well as the recent published state-of-the-art trackers: DiffusionTrack [52], MixFormer [13], OTrack [53] and D3Sv2 [15]. The proposed DAM4SAM outperforms the VOT2022 winner MS_AOT by a significant margin of 12% in EAO. Note that the performance improvement is a consequence of both improved accuracy (2%) and robustness (3%) compared to the MS_AOT [2]. In addition to achieving state-of-the-art performance, DAM4SAM outperforms also the baseline SAM2.1 by a significant margin of 9% EAO.

The results on both, VOT2020 and VOT2022 clearly show that DAM4SAM outperforms all trackers, including the challenges top-performers and the recently published trackers, setting new state-of-the-art on these benchmarks. Despite its simplicity, the proposed *training-free* memory management is the key element for achieving excellent tracking performance.

Table 6. State-of-the-art comparison on the VOT2022 benchmark. The challenge winner is marked by ✂.

	EAO	Acc.	Rob.
MS_AOT ✂ (NeurIPS21)	0.673 ③	0.781	0.944 ③
DiffusionTrack (AAAI24)	0.634	-	-
DAMTMask (ECCVW22)	0.624	0.796 ③	0.891
MixFormer (CVPR22)	0.589	0.799 ②	0.878
OTrack (ECCV22)	0.581	0.775	0.867
Linker (TMM24)	0.559	0.772	0.861
SRATransT (ECCVW22)	0.547	0.743	0.866
TransT (CVPR21)	0.542	0.743	0.865
GDFormer (ECCVW22)	0.538	0.744	0.861
TransLL (ECCVW22)	0.530	0.735	0.861
LWL_B2S (ECCVW20)	0.516	0.736	0.831
D3Sv2 (TPAMI21)	0.497	0.713	0.827
SAM2.1 (ICLR25)	0.692 ②	0.779	0.946 ②
DAM4SAM	0.753 ①	0.800 ①	0.969 ①

VOTS2024 benchmark [9] is one of the most recent tracking benchmarks, consisting of 144 sequences. In contrast to VOT2020 and VOT2022, the VOTS2024 benchmark introduces a new, larger dataset, tracking multiple objects in the same scene (with ground truth sequestered at an evaluation server) and a new performance measure, designed to address short-term, long-term⁴, single and multi-target tracking scenarios. The VOTS2024 is currently considered as the most challenging tracking benchmark.

Although each sequence might contain several targets to track, the benchmark allows either to track these individually or jointly. The VOTS performance measures do not distinguish between how the tracks of all required targets are obtained, thus generalizing over both tracking approaches. As DAM4SAM is a single-target tracker, the targets are tracked individually in the VOTS2024 benchmark.

Results are reported in Table 7. It is worth pointing out that the top performers are mostly unpublished (not peer-reviewed) trackers, tuned for the competition, and often complex ad-hoc combinations of multiple methods. For example, the challenge winner S3-Track combines visual and (mono)depth features, uses several huge backbones, and is much more complex than DAM4SAM. Despite this, using the same parameters as in other experiments, DAM4SAM achieves a solid second place among the challenging VOTS2024 competitors. In particular, it outperforms the challenge-tuned versions of the recently published trackers LORAT [3], Cutie [17], and the VOT2022 winner AOT [2]. Furthermore, the proposed memory management mechanism in DAM4SAM contributes to 8% performance boost in tracking quality compared to the baseline SAM2.1, mostly due to 9% higher robustness.

VOT2022-RT benchmark [8] was a challenge specifically designed for real-time evaluation. The sequences remain the same as in previously mentioned VOT2022. Specifically, VOT challenges are run by VOT toolkits, which manage the real-time constraints. A frame is sent to the tracker, which needs to process it and report the target position at 20FPS frame rate. If the tracker is not able to process the frame in time, the prediction from the previous frame is used as the estimate for the current frame and next frame is sent to the tracker. Such a setup simulates actual real-time scenarios, which is much more realistic than reporting just the average tracking speed. The tracking performance is measured using standard VOT2022 measures [8]: the primary

⁴The target may leave the field of view and re-appear later in the video.

Table 7. State-of-the-art comparison on the VOT2024 benchmark. The challenge winner is marked by ⚡.

	Quality	Acc.	Rob.
S3-Track ⚡ (arXiv23)	0.722 ①	0.784	0.889 ①
DMAOT_SAM (NeurIPS22)	0.653	0.794 ①	0.780
HQ-DMAOT (NeurIPS22)	0.639	0.754	0.790
DMAOT (NeurIPS22)	0.636	0.751	0.795 ③
LY-SAM (ECCVW24)	0.631	0.765	0.776
Cutie (CVPR24)	0.607	0.756	0.730
AOT (NeurIPS21)	0.550	0.698	0.767
LORAT (ECCV24)	0.536	0.725	0.784
SAM2.1 (ICLR25)	0.661 ③	0.791 ③	0.790
DAM4SAM	0.711 ②	0.793 ②	0.864 ②

measure expected average overlap (EAO), and two auxiliary measures, i.e., accuracy and robustness.

The results in Table 8 show that the proposed DAM4SAM (L model size) outperforms all trackers that participated in VOT2022-RT challenge. In particular, it outperforms the challenge winner by 4% in EAO demonstrating excellent real-time performance. These results show that the proposed distractor-aware memory adds only a small computational overhead, yet bringing remarkable robustness capabilities and making it useful for real applications.

Table 8. Real-time performance on VOT2022-RT challenge. The challenge winner is marked by ⚡.

	EAO	Acc.	Rob.
MS_AOT ⚡ (NeurIPS21)	0.610 ③	0.751 ②	0.921 ③
OTrack (ECCV22)	0.569	0.766 ①	0.860
SRATransT (ECCVW22)	0.547	0.743 ③	0.866
SAM2.1 (ICLR25)	0.614 ②	0.722	0.922 ②
DAM4SAM	0.635 ①	0.717	0.942 ①

4.4. Comparison on box datasets. For a complete evaluation, we compare DAM4SAM on the following three standard bounding box tracking datasets: LaSoT [4], LaSoT_{ext} [10] and GoT10k [5]. Since frames are annotated by bounding boxes and SAM2 requires a segmentation mask provided in the first frame, we use the same SAM2 model to estimate the initialization mask. The min-max operation is applied on the predicted masks to obtain the axis-aligned bounding boxes required for the evaluation. Tracking performance is computed using area-under-the-success-rate curve [7] (AUC) in LaSoT [4] and LaSoT_{ext} [10] and the average overlap [5] (AO) in Got10k.

LaSoT [4] is a large-scale tracking dataset with 1,400 video sequences, with 280 evaluation sequences and the rest are used for training. The sequences are equally split into 70 categories, where each category is represented by 20 sequences (16 for training and 4 for evaluation). The dataset consists of various scenarios covering short-term and long-term tracking. Results are shown in Table 9. The proposed DAM4SAM outperforms the baseline SAM2.1 [22] by 7.3%, which indicates that the proposed memory management is important also in a bounding box tracking setup. Furthermore, DAM4SAM performs on par with the top-performing tracker LORAT [3], which was tuned on LaSoT training set, i.e., on the categories included in the evaluation set. It is worth noting that LORAT has approximately 50% more training parameters than DAM4SAM, making the model significantly more complex.

For additional insights on tracking performance, DAM4SAM is evaluated on fourteen visual attributes of the LaSoT dataset and compared with the baseline SAM2.1 in Table 10. The results show that the most challenging attributes are fast motion and full occlusion, which goes in line with the general findings of the past public attribute-based analyses [45, 54]. Interestingly, DAM4SAM significantly outperforms SAM2.1 on these attributes, which demonstrates the effectiveness of the proposed design. Specifically, the proposed update rules (where a frame is added to the DRM only when tracking is deemed confident and a distractor is present, and updates are suppressed when the target is absent) are key to the improved re-detection capability of the tracker. This mechanism directly contributes to the observed performance gains under the full occlusion attribute. Moreover, fast-moving targets are more likely to encounter nearby distractors, making effective distractor resolution particularly critical in such scenarios. Consequently,

the proposed distractor-aware memory update plays an important role in the improved tracking performance for fast motion scenarios.

On the other hand, the least challenging attributes are camera motion and motion blur, while the attributes for which the improvement is the smallest, are partial occlusion, rotation and deformation. The results indicate that these attributes are already well addressed by the baseline method.

LaSoT_{ext} [10] is an extension of the LaSoT [4] dataset, by 150 test sequences, divided into 15 new categories, which are not present in the training dataset. The results in Table 9 show that DAM4SAM outperforms the baseline version by a comfortable margin of 7% in AUC. In addition, it outperforms the second-best tracker LORAT [3] by 7.6%. This indicates that DAM4SAM generalizes well across various object categories while existing trackers suffer a much larger performance drop.

GoT10k [5] is another widely used large-scale tracking dataset, composed of $\sim 10k$ video sequences, from which 180 sequences are used for testing. We observed that top-performing trackers on GoT10k test set achieve excellent tracking performance, e.g., more than 78% of average overlap, which leaves only small room for potential improvements. However, a solid 3.7% boost in tracking performance is observed when comparing DAM4SAM to the top-performers LORAT [3] and ODTrack [16]. A close inspection of the DAM4SAM results reveals that more than 99% of frames are successfully tracked (i.e., with a non-zero overlap), which indicates that GoT10k [5] difficulty level is diminishing for modern trackers.

Table 9. State-of-the-art comparison on three standard bounding-box benchmarks: LaSoT, LaSoT_{ext}, and GoT10k.

	LaSoT (AUC)	LaSoT _{ext} (AUC)	GoT10k (AO)
MixViT (TPAMI24)	72.4	-	75.7
LORAT (ECCV24)	75.1 ①	56.6 ③	78.2 ③
ODTrack (AAAI24)	74.0 ②	53.9	78.2 ③
DiffusionTrack (AAAI24)	72.3	-	74.7
DropTrack (CVPR23)	71.8	52.7	75.9
SeqTrack (CVPR23)	72.5 ③	50.7	74.8
MixFormer (CVPR22)	70.1	-	71.2
GRM-256 (CVPR23)	69.9	-	73.4
ROMTrack (ICCV23)	71.4	51.3	74.2
OTrack (ECCV22)	71.1	50.5	73.7
KeepTrack (CVPR21)	67.1	48.2	-
TOMP (CVPR22)	68.5	-	-
SAM2.1 (ICLR25)	70.0	56.9 ②	80.7 ②
DAM4SAM	75.1 ①	60.9 ①	81.1 ①

Table 10. Per-attribute comparison of SAM2.1 and DAM4SAM on LaSoT dataset. The AUC denotes overall performance on the whole dataset in terms of area-under-the-curve measure. The attribute abbreviations are the following: ARC (aspect ratio change), BC (background clutter), CM (camera motion), DEF (deformation), FM (fast motion), FOC (full occlusion), IV (illumination variation), LR (low resolution), MB (motion blur), OV (out-of-view), POC (partial occlusion), ROT (rotation), SV (scale variation) and VC (viewpoint change).

	AUC	ARC	BC	CM	DEF	FM	FOC	IV	LR	MB	OV	POC	ROT	SV	VC
SAM2.1 (ICLR25)	70.0	68.8	65.3	70.7	71.7	60.2	60.6	65.9	61.5	69.2	63.3	69.3	68.2	69.5	62.9
DAM4SAM	75.1	74.1	70.8	79.8	76.1	68.3	68.7	75.1	69.2	77.2	70.2	73.4	72.2	74.8	73.6
Improvement (%)	7.3	7.7	8.4	12.9	6.1	13.5	13.4	14	12.5	11.6	10.9	5.9	5.9	7.6	17

Additionally, we evaluate the tracker’s performance on three less commonly used benchmarks: OTB100 [7], NFS [55], and UAV123 [56]. The results are summarized in Table 11. Across all three benchmarks, DAM4SAM demonstrates a performance improvement over the baseline SAM2.1 [22], with gains ranging from 1.5% to 3%. On the OTB100 [57] dataset specifically, DAM4SAM achieves a tracking robustness rate of 99.5%, outperforming LORAT [3], which reports 96.5%.

4.5. Comparison on VOS datasets. To further assess the effectiveness of distractor-aware memory, we compare DAM4SAM with state-of-the-art methods across three video object segmentation benchmarks: VOST [58], LVOS v1 [23], and LVOS v2 [27]. We report results on the semi-supervised VOS task. Methods are initialized with the ground truth segmentation mask and the predictions are evaluated with the $J\&F$ metric, an average of the VOS metrics J (region similarity) and F (contour accuracy). Note that VOS

Table 11. State-of-the-art comparison on three additional bounding box benchmarks.

	OTB100 (AO)	NFS (AUC)	UAV123 (AUC)
MixFormer (CVPR22)	70.7 ③	-	70.4
SeqTrack (CVPR23)	68.3	66.2	68.5
LORAT (ECCV24)	72.0 ①	66.7 ③	72.5 ①
SAM2.1 (ICLR25)	70.6	67.6 ②	68.8 ③
DAM4SAM	71.7 ②	68.6 ①	70.9 ②

benchmarks typically have a lower frame rate (5fps) compared to tracking datasets (30fps). To adapt the distractor-aware memory to this, we reduce the memory stride for both the RAM and DRM components from 5 to 1. This ensures that the memory updates created for tracking scenes align with the sparser annotation frequency in the VOS setting.

VOST benchmark [58] is designed for video object segmentation under complex object transformations. Unlike other datasets, VOST features objects that are broken, torn, or reshaped into entirely new forms, significantly altering their appearance. The validation set consists of 70 sequences, each averaging 21 seconds in length. Results are shown in Table 12. DAM4SAM outperforms the baseline SAM2.1 [22] by 4.3%, highlighting that the distractor-aware memory effectively enhances performance even in scenes where objects undergo significant transformations.

LVOS datasets [23, 27] are long-term video object segmentation benchmarks. LVOS v2 [27] is an extended version of LVOS v1, increasing the number of sequences in the validation set from 50 to 140. On average, sequences span 68 seconds. The result in Table 12 show that DAM4SAM outperforms the baseline SAM2.1 [22] by 2.7% and 6.1%, on LVOS v1 and v2, respectively.

Table 12. State-of-the-art comparison on video object segmentation benchmarks. Table reports $J&F$ scores on validation sets.

	VOST	LVOSv1	LVOSv2
RDE (CVPR22)	—	—	62.2
DeAOT (NeurIPS22)	—	—	63.9
DEVA (ICCV23)	—	55.9	—
DDMemory (ICCV23)	—	60.7	—
XMem++ (ICCV23)	37.5	—	61.6
Cutie (CVPR24)	40.8 ③	66.0 ③	71.2 ③
SAM2.1 (ICLR25)	53.1 ②	80.1 ②	80.6 ②
DAM4SAM	55.4 ①	82.8 ①	85.5 ①

4.6. Qualitative analysis. Figure 7 presents qualitative results of DAM4SAM on several sequences from the DiDi dataset, illustrating its performance in the presence of challenging distractors. Across all sequences, DAM4SAM successfully tracks the target object without failure. It demonstrates strong robustness in crowded scenes, as seen in the first two examples. In the third sequence, where the task is to localize a turtle amidst multiple distractors and a background with similar appearance, DAM4SAM maintains accurate tracking. In the final example, taken from a basketball game, the model continues to perform reliably even with a fast-moving target.

Figure 8 presents a qualitative comparison between the baseline SAM2.1 and the proposed DAM4SAM on four video sequences from the DiDi dataset. In the first row, a zebra is tracked with other zebras in its vicinity. When the zebra is partially occluded, SAM2.1 drifts to the wrong zebra and starts to track it, while DAM4SAM tracks only the visible part of the target during occlusion and stays on the selected zebra until the end of the sequence.

In the second row of Figure 8, the baseline SAM2.1 tracker successfully tracks the bus until the full occlusion and fails to re-detect it after the re-appearance. This failure occurs due to the too frequent memory updates when target is occluded and is successfully addressed with the proposed memory update in DAM4SAM.

The third row in Figure 8 shows tracking of a flamingo’s head. The baseline SAM2.1 tends to jump on the bird’s beak or extend to the whole body, since it prefers to segment the regions with so-called high objectness (i.e., regions with well-defined edges). The proposed DAM4SAM successfully tracks the flamingo’s head even if the edge between the head and the neck is not clearly visible. In this case, part of the neck is segmented by

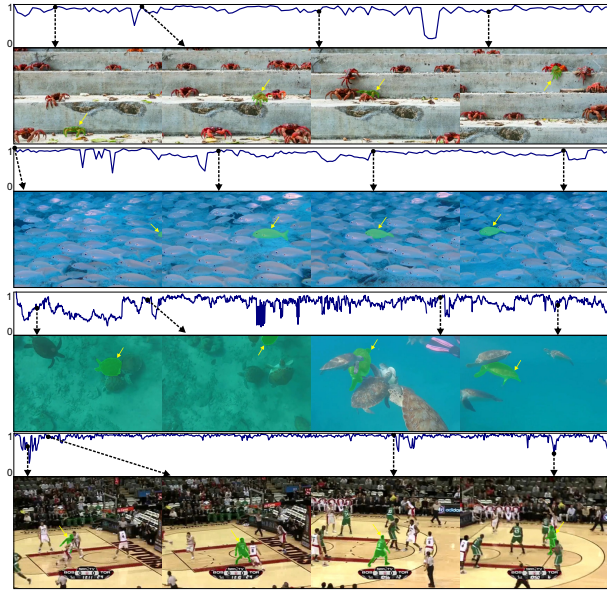


Figure 7. DAM4SAM qualitative results on the DiDi dataset with predicted masks shown in green, and tracked objects denoted by arrows. Per-frame overlaps (intersection-over-union for each frame) are shown above the figures to indicate failure-free tracking over the entire sequence.

an alternative mask and thus detected as a distractor. Updating the distractor resolving memory (DRM) using such *critical* frames results in a more stable and accurate tracking.

A similar effect is demonstrated in the fourth row of Figure 8, SAM2.1 starts tracking the wrong object after the tracked fish is occluded by another. On the other hand, DAM4SAM successfully detects such critical frames, updates the DRM and avoids the tracking failure.



Figure 8. Qualitative comparison of the baseline SAM2.1 (blue) and the proposed DAM4SAM (yellow). The symbol $\approx$ denotes approximately identical outputs and $\emptyset$ denotes an empty prediction (i.e., mask with all-zeros). Tracked object is denoted with a green arrow.

4.7. Failure cases. An analysis of failure cases of the proposed tracker is presented in Figure 9. We identify two primary reasons why DAM4SAM fails. The first is the presence of extreme distractors, where the background closely resembles the target. One such example is shown in Figure 9(a). Although DAM4SAM successfully tracks the target through nearly half of the sequence, it eventually loses it due to the high visual similarity between the white rabbit (the target) and the avalanche in the background. It is worth noting that such cases are highly challenging even for human observers.

The second category of failure are generalizations to larger parts of the object, which we refer to as internal distractors. While these failures may significantly affect tracking accuracy, they do not compromise robustness. Examples are shown in Figure 9(b, c). A closer examination reveals that these issues may come from SAM2’s limitation to generalize to large objects and its tendency to produce multiple plausible masks – a behavior likely influenced by its training under uncertainty. Such behavior is typically observed when tracking object parts with ambiguous or poorly defined boundaries, where the extracted target features are insufficient to discriminate from the background. For instance, in Figure 9(b), the boundary between the palm and the forearm is ambiguous. Combined with rapid appearance and pose variations, this reduces separability between

the target region and the internal distractor, leading to over-segmentation and eventual tracking failure. Addressing these failure cases likely requires improvements to the feature encoding and representation, rather than adjustments to the memory manipulation mechanism alone. We experimented with prompting SAM2 on the failure frame (where DAM4SAM overgeneralizes) using the first frame and its ground truth mask; the model would predict an overgeneralized mask, suggesting the error likely originates from SAM2 segmentation methodology itself.

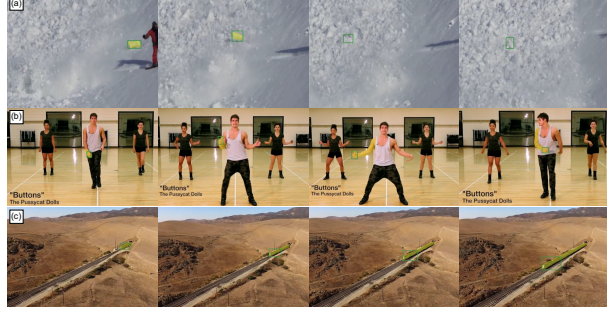


Figure 9. Failure cases of DAM4SAM on DiDi dataset. The two most common reasons for failure are: (a) extreme visual similarity between the target and the background, and (b, c) overgeneralization to larger parts of the object when the goal is to track only a part of it. Yellow masks denote tracker predictions and green boxes are groundtruth.

5. Generalization analysis

In this section, we present experiments designed to investigate the generalization capabilities of the proposed drop-in memory. We begin by integrating DAM into different SAM2 [22] model sizes and versions, presented in Section 5.1 and Section 5.2, respectively. To further demonstrate the effectiveness of distractor-aware memory when applied to a different tracker, Section 5.3 presents results from integrating DAM into EfficientTAM [28] and Section 5.4 presents results when integrated into EdgeTAM [59].

5.1. Model size. The Segment Anything Model 2 (SAM2) [22] was originally developed in four model sizes, denoted by tiny (T), small (S), base (B) and large (L). In Table 13 these four model sizes of unchanged SAM2.1 are compared with our DAM4SAM version, presented in the paper on the new DiDi dataset. Results show a clear and consistent performance improvement across all four model sizes. In particular, the tracking quality improves by approximately 6% or 7%, depending on the model size, mostly due to the improved robustness. These results show that the proposed distractor-aware memory generalizes well over various model sizes, demonstrating that it is not overfitted to the exact SAM2 model.

Beyond overall tracking accuracy, Table 13 also reports the impact of the proposed distractor-aware memory on average tracking performance, measured in frames per second (FPS). Across different model sizes, the speed reduction is relatively consistent, ranging from 10% to 14% compared to the original SAM2.1. The majority of this overhead arises from the additional computations required to identify anchor DRM distractor frames, including alternative mask processing and IoU evaluations between selected and alternative masks. We argue that the 10-14% increase in computational cost is justified by the corresponding 6-7% improvement in tracking performance. A more detailed real-time analysis is provided in Section 4.3, corresponding to the VOT2022-RT challenge.

Table 13. Comparison between SAM2.1 (SAM) and DAM4SAM (D4S) with different model sizes on DiDi dataset. The T, S, B, L denote the tiny, small, base and large Hiera backbone sizes, respectively. Param denotes number of parameters, Mem memory footprint during inference in gigabytes, FPS inference speed in frames-per-second, while Acc. and Rob. denote accuracy and robustness, respectively.

	Param	Mem	FPS	Quality	Acc.	Rob.
SAM-T	39M	0.63	17.4	0.600	0.697	0.848
D4S-T	39M	0.63	15.2	0.642	0.695	0.907
SAM-S	46M	0.68	17.0	0.630	0.718	0.866
D4S-S	46M	0.68	15.3	0.668	0.709	0.930
SAM-B	81M	0.90	16.0	0.624	0.721	0.856
D4S-B	81M	0.90	13.7	0.664	0.709	0.930
SAM-L	224M	1.78	13.2	0.649	0.720	0.887
D4S-L	224M	1.78	11.8	0.694	0.727	0.944

5.2. Model version. This section compares the performance improvements for the two individual SAM versions, i.e., SAM2 and SAM2.1. SAM2.1 version improves the initial version in handling small and visually similar objects by introducing additional augmentation techniques in training. It also includes improved occlusion handling by training the model on longer frame sequences.

Results are shown in Table 14. To demonstrate the improvement of the 2.1 model version over version 2, we compare SAM2 with SAM2.1 on the DiDi dataset, which results in approximately 3.5% improvement in tracking quality. Next, we compare the original SAM2 and the version with our new memory model. The tracking performance improves by 7%, which is well beyond the performance improvement from SAM2 to SAM2.1 and supports the importance of a high-quality memory model and the memory management regime.

A similar performance boost (close to 7%) is observed between SAM2.1 and DAM4SAM, which implies complementarity of the new memory model with the baseline method performance improvements, which come from better training. Similarly as in Section 5.1, we conclude that the proposed distractor-aware memory is robust to different model versions, demonstrating a consistent improvements in tracking performance on two SAM2 versions.

Table 14. Comparison of SAM2 and SAM2.1 on DiDi dataset.

	Quality	Acc.	Rob.
SAM2 (ICLR25)	0.627	0.723	0.850
DAM4SAM (2)	0.668 $\uparrow$ 7%	0.710	0.929
SAM2.1 (ICLR25)	0.649	0.720	0.887
DAM4SAM (2.1)	0.694 $\uparrow$ 7%	0.727	0.944

5.3. Lightweight architecture. To further validate the advantages of the proposed distractor-aware memory, we integrate it into a recent tracking model, EfficientTAM [28], an efficient variant of SAM2 [22]. The authors propose using a lightweight image encoder to achieve approximately $2\times$ speedup and $2.4\times$ parameter reduction (34M compared to 81M), while maintaining performance comparable to SAM2-B model. The memory mechanism follows the principles of SAM2, with the memory bank storing the last six frames, along with the initialization frame, regardless of target visibility.

The modified model, EfficientTAM with integrated DAM, is referred to as DAM4TAM. A performance comparison on the DiDi dataset is presented in Table 15. DAM4TAM achieves a 12% improvement in overall quality over EfficientTAM on the DiDi dataset, due to a substantial increase in tracking robustness. Although initial comparisons between SAM2.1-S and EfficientTAM-S show a significant quality gap (with the latter performing 4.3% worse), integrating DAM into both models reduces this gap to just 1.3%. DAM4TAM runs at 25 fps, which is comparable to the speed of TAM-S.⁵

Table 15. Comparison of different versions of SAM2, EfficientTAM and EdgeTAM with the proposed DAM variants on DiDi.

	Quality	Acc.	Rob.
EfficientTAM-S (arXiv24)	0.587	0.706	0.821
DAM4TAM-S	0.653 $\uparrow$ 11%	0.704	0.915
EdgeTAM (CVPR25)	0.566	0.673	0.821
DAM4EdgeTAM	0.591 $\uparrow$ 4%	0.669	0.858
SAM2.1-S (ICLR25)	0.630	0.718	0.866
DAM4SAM-S	0.668 $\uparrow$ 6%	0.709	0.930
SAM2.1-L (ICLR25)	0.649	0.720	0.887
DAM4SAM-L	0.694 $\uparrow$ 7%	0.727	0.944

DAM4TAM is evaluated against the baseline EfficientTAM [28] on VOT2022 tracking dataset [8] and two video object segmentation datasets: VOST [58] and LVOSv2 [27]. The results are presented in Table 16. On VOT2022, DAM4TAM improves baseline’s performance by 8%, mostly due to a robustness score of 0.938, while using $6.5\times$ fewer parameters than SAM2.1-L (which has a robustness score of 0.946). On VOST, DAM4TAM outperforms EfficientTAM by 5% in EAO, again achieving result comparable to SAM2.1-L.

⁵All trackers were evaluated on a machine equipped with an AMD EPYC 7763 64-Core 2.45 GHz CPU and an NVIDIA A100 40GB GPU.

Table 16. Comparison of baseline and DAM4TAM on VOT2022, VOST, and LVOSv2 datasets.

	VOT2022 (EAO)	VOST ($J\&F$)	LVOSv2 ($J\&F$)
EfficientTAM-S (arXiv24)	0.585	0.509	0.805
DAM4TAM-S	0.630 $\uparrow$ 8%	0.537 $\uparrow$ 6%	0.817 $\uparrow$ 1.5%

5.4. Edge architecture. Similar to EfficientTAM [28], a recent approach called EdgeTAM [59] was proposed to enable SAM2-based models to run on mobile devices while maintaining comparable performance. The authors identify that, in addition to the image encoder, the memory attention blocks are a major latency bottleneck. To address this, they introduce a 2D spatial perceiver module that significantly reduces computational cost. EdgeTAM runs with 15 FPS on an iPhone, compared to only 0.7 FPS for SAM2. For the memory management, EdgeTAM follows the same principles as SAM2.

The modified version with integrated DAM is referred to as DAM4EdgeTAM. Table 15, compares DAM4EdgeTAM to the baseline on the DiDi dataset. The DAM-enhanced model achieves improved performance, due to an approximate 4% better robustness on the dataset.

6. Conclusion

We proposed a drop-in, training-free Distractor-Aware Memory (DAM) model and a corresponding memory management regime for memory-based trackers. The DAM architecture separates memory by tracking functionality into two components: Recent Appearances Memory (RAM) for maintaining segmentation accuracy, and Distractor-Resolving Memory (DRM) for enhancing robustness by supporting tracking recovery after failure and re-detection. We introduce efficient, introspection-based update rules for both memory types, relying solely on the tracker’s outputs without additional training.

For fair performance evaluation in scenes with distractors, we construct DiDi, a distractor distilled dataset consisting of 180 video sequences with an average length of 1,500 frames. The dataset construction process (and manual inspection) ensure presence of distractors in each video, which makes DiDi a unique tracking benchmark. Since distractors are one of key tracking challenges, leading to tracking failures, DiDi has a strong potential to facilitate development of future distractor-robust trackers.

DAM is implemented on top of SAM2.1 [22], forming DAM4SAM. Extensive analysis and ablation studies support our design choices. Using frozen SAM2.1-L weights, DAM4SAM achieves state-of-the-art performance on ten benchmarks, with only a moderate 10% speed reduction compared to SAM2.1. Tracking speed is not affected by the number of distractors in the scene. On the other hand, more distractors in general may lead to more potential points of failure. Nevertheless, we did not observe a correlation between a number of distractors and low tracking performance.

DAM is integrated into multiple versions (v2 and v2.1) and sizes (tiny, small, base, and large) of SAM2 model, consistently achieving a 6–7% improvement on the DiDi dataset. For efficient tracking we integrated DAM into two recent SAM-based architectures: Efficient Track Anything [28] and EdgeTAM [59]. The DAM-enhanced versions consistently outperform the baseline on multiple tracking and segmentation benchmarks. These results indicate that DAM4SAM is practically useful in all standard tracking applications, such as surveillance, video editing and sports analytics, and extends to realtime and even edge-device tracking setups. Since it is able to provide precise object segmentation (not just an approximate object position as is the case for bounding box trackers), the application spectrum is potentially wider.

Failure case analysis of DAM4SAM reveals unresolved ambiguities in the foundational model SAM2 [22], regarding object boundaries. This issue likely comes from training on segmentation datasets, rather than tracking-specific data. SAM2 exhibits a tendency to favor objects with clear boundaries, which can lead to inconsistencies when tracking only part of an object.

Experimental results suggest that more research should focus on efficient memory designs, with possibly learnable management methods. Such formulation includes training reinforcement learning policies to determine when a frame should be added to which part of the memory, given different observations such as visual information, predicted mask, alternative masks, tracking uncertainty, and the current memory content. In fact, following the DAM4SAM conference publication [29], a preliminary work [60] emerged attempting to follow this avenue. Moreover, further performance gains are expected to come from improved foundation models. For example, a straight-forward replacement of

the memory with DAM in the just published SAM3 foundation model [61] improves its overall tracking performance by 1.5% in tracking quality on DiDi.

Finally, we hypothesize that DAM could positively impact the general and classical multi object tracking (GMOT/MOT) research, which typically considers tracking many similar objects. Notably, DAM's resilience to distractors could lead to better individual object frame-to-frame association, considering the other objects as distractors. MOT formulation as multiple independent robust single-object trackers would indeed challenge the established paradigm. Nevertheless, extensions that would rival the modern trackers will more likely require in-depth integration with the latest trainable association modules [62–64] emerging from the MOT domain. We expect these avenues will lead to further advancements, perhaps facilitating methodological convergence of single and multi object trackers.

ACKNOWLEDGEMENT. This work was supported by Slovenian research agency program P2-0214 and projects J2-2506, Z2-4459 and J2-60054, and by supercomputing network SLING (ARNES, EuroHPC Vega - IZUM).

Bibliography

- Chen X et al. (2021) Transformer tracking in *Comp. Vis. Patt. Recognition*. pp. 8126–8135.
- Yang Z, Wei Y, Yang Y (2021) Associating objects with transformers for video object segmentation. *Neural Inf. Proc. Systems* 34:2491–2502.
- Lin L et al. (2025) Tracking meets lora: Faster training, larger model, stronger performance in *Proc. European Conf. Computer Vision*. pp. 300–318.
- Fan H et al. (2019) Lasot: A high-quality benchmark for large-scale single object tracking in *Comp. Vis. Patt. Recognition*.
- Huang L, Zhao X, Huang K (2018) Got-10k: A large high-diversity benchmark for generic object tracking in the wild in *IEEE Trans. Pattern Anal. Mach. Intell.* Vol. 43, pp. 1562–1577.
- Muller M, Bibi A, Giancola S, Alsabaihi S, Ghanem B (2018) TrackingNet: A large-scale dataset and benchmark for object tracking in the wild in *Proc. European Conf. Computer Vision*.
- Wu Y, Lim J, Yang MH (2015) Object tracking benchmark in *IEEE Trans. Pattern Anal. Mach. Intell.* Vol. 37, pp. 1834–1848.
- Kristan M et al. (2022) The tenth visual object tracking vot2022 challenge results in *Proc. European Conf. Computer Vision Workshops*. pp. 431–460.
- Kristan M et al. (2024) The second visual object tracking segmentation vots2024 challenge results.
- Fan H et al. (2021) Lasot: A high-quality large-scale single object tracking benchmark in *Int. J. Comput. Vision*. Vol. 129, pp. 439–461.
- Bertinetto L, Valmadre J, Henriques JF, Vedaldi A, Torr PHS (2016) Fully-convolutional siamese networks for object tracking in *Proc. European Conf. Computer Vision Workshops*. pp. 850–865.
- Yan B, Peng H, Fu J, Wang D, Lu H (2021) Learning spatio-temporal transformer for visual tracking in *Int. Conf. Computer Vision*. pp. 10448–10457.
- Cui Y, Jiang C, Wang L, Wu G (2022) Mixformer: End-to-end tracking with iterative mixed attention in *Comp. Vis. Patt. Recognition*. pp. 13608–13618.
- Chen X, Peng H, Wang D, Lu H, Hu H (2023) Seqtrack: Sequence to sequence learning for visual object tracking in *Comp. Vis. Patt. Recognition*. pp. 14572–14581.
- Lukežič A, Matas J, Kristan M (2021) A discriminative single-shot segmentation network for visual object tracking in *IEEE Trans. Pattern Anal. Mach. Intell.* Vol. 44, pp. 9742–9755.
- Zheng Y et al. (2024) Odtrack: Online dense temporal token learning for visual tracking in *Proc. of the AAAI Conf. on Artificial Intelligence*. Vol. 38, pp. 7588–7596.
- Cheng HK, Oh SW, Price B, Lee JY, Schwing A (2024) Putting the object back into video object segmentation in *Comp. Vis. Patt. Recognition*. pp. 3151–3161.
- Lukežič A, Trojer Ž, Matas J, Kristan M (2024) A new dataset and a distractor-aware architecture for transparent object tracking. *Int. J. Comput. Vision* pp. 1–14.
- Kristan M et al. (2023) The first visual object tracking segmentation vots2023 challenge results in *Int. Conf. Computer Vision Workshops*. pp. 1796–1818.
- Cheng HK, Schwing AG (2022) Xmem: Long-term video object segmentation with an atkinson-shiffrin memory model in *Proc. European Conf. Computer Vision*. pp. 640–658.
- Zhou J, Pang Z, Wang YX (2024) Rmem: Restricted memory banks improve video object segmentation in *Comp. Vis. Patt. Recognition*. pp. 18602–18611.
- Ravi N et al. (2025) Sam 2: Segment anything in images and videos in *Proc. Int. Conf. Learning Representations*.
- Hong L et al. (2023) Lvos: A benchmark for long-term video object segmentation in *Int. Conf. Computer Vision*. pp. 13480–13492.
- Xu N et al. (2018) Youtube-vos: A large-scale video object segmentation benchmark. *arXiv preprint arXiv:1809.03327*.
- Ding H et al. (2023) Mose: A new dataset for video object segmentation in complex scenes in *Comp. Vis. Patt. Recognition*. pp. 20224–20234.
- Pont-Tuset J et al. (2017) The 2017 davis challenge on video object segmentation. *arXiv:1704.00675*.
- Hong L et al. (2024) Lvos: A benchmark for large-scale long-term video object segmentation. *arXiv preprint arXiv:2404.19326*.
- Xiong Y et al. (2024) Efficient track anything. *arXiv preprint arXiv:2411.18933*.
- Videnović J, Lukežič A, Kristan M (2025) A distractor-aware memory for visual object tracking with SAM2 in *Comp. Vis. Patt. Recognition*.
- Nam H, Han B (2016) Learning multi-domain convolutional neural networks for visual tracking in *Comp. Vis. Patt. Recognition*. pp. 4293–4302.
- Danelljan M, Bhat G, Khan FS, Felsberg M (2019) ATOM: Accurate tracking by overlap maximization in *Comp. Vis. Patt. Recognition*.
- Henriques JF, Caseiro R, Martins P, Batista J (2014) High-speed tracking with kernelized correlation filters in *IEEE Trans. Pattern Anal. Mach. Intell.* Vol. 37, pp. 583–596.
- Li Y, Zhu J (2014) A scale adaptive kernel correlation filter tracker with feature integration in *Proc. European Conf. Computer Vision Workshops*. pp. 254–265.
- Li B, Yan J, Wu W, Zhu Z, Hu X (2018) High performance visual tracking with siamese region proposal network in *Comp. Vis. Patt. Recognition*.
- Li B et al. (2019) SiamRPN++: Evolution of siamese visual tracking with very deep networks in *Comp. Vis. Patt. Recognition*.
- Guo D, Wang J, Cui Y, Wang Z, Chen S (2020) Siamcar: Siamese fully convolutional classification and regression for visual tracking in *Comp. Vis. Patt. Recognition*.
- Chen B et al. (2022) Backbone is all your need: A simplified architecture for visual object tracking in *Proc. European Conf. Computer Vision*. pp. 375–392.
- Ding S et al. (2024) Sam2long: Enhancing sam 2 for long video segmentation with a training-free memory tree. *arXiv preprint arXiv:2410.16268*.
- Yang CY, Huang HW, Chai W, Jiang Z, Hwang JN (2024) Samurai: Adapting segment anything model for zero-shot visual tracking with motion-aware memory. *arXiv:2411.11922*.
- Cuttano C, Trivigno G, Rosi G, Masone C, Averta G (2025) Samwise: Infusing wisdom in sam2 for text-driven video segmentation in *Comp. Vis. Patt. Recognition*.
- Mayer C, Danelljan M, Paudel DP, Van Gool L (2021) Learning target candidate association to keep track of what not to track in *Comp. Vis. Patt. Recognition*. pp. 13444–13454.
- Kirillov A et al. (2023) Segment anything in *Int. Conf. Computer Vision*. pp. 4015–4026.
- Ryali C et al. (2023) Hiera: A hierarchical vision transformer without the bells-and-whistles in *Proc. Int. Conf. Mach. Learning*. pp. 29441–29454.
- Alawode B et al. (2022) Utb180: A high-quality benchmark for underwater tracking in *Proc. Asian Conf. Computer Vision*. pp. 3326–3342.
- Kristan M et al. (2020) The eighth visual object tracking VOT2020 challenge results in *Proc. European Conf. Computer Vision Workshops*. pp. 547–601.
- Oquab M et al. (2023) Dinov2: Learning robust visual features without supervision. *arXiv:2304.07193*.
- Xie J et al. (2024) Autoregressive queries for adaptive tracking with spatio-temporal transformers in *Comp. Vis. Patt. Recognition*. pp. 19300–19309.
- Kristan M et al. (2016) A novel performance evaluation methodology for single-target trackers in *IEEE Trans. Pattern Anal. Mach. Intell.* Vol. 38, pp. 213–215.
- VOTS (2025) VOT challenge (<https://www.votchallenge.net>).

Accessed: 2025-08-11.

50. Cui Y, Jiang C, Wu G, Wang L (2024) Mixformer: End-to-end tracking with iterative mixed attention. *IEEE Trans. Pattern Anal. Mach. Intell.* 46(6):4129–4146.
51. Ma Z, Wang L, Zhang H, Lu W, Yin J (2020) Rpt: Learning point set representation for siamese visual tracking in *Proc. European Conf. Computer Vision Workshops*. pp. 653–665.
52. Luo R et al. (2024) Diffusiontrack: Diffusion model for multi-object tracking in *Proc. of the AAAI Conf. on Artificial Intelligence*. Vol. 38, pp. 3991–3999.
53. Ye B, Chang H, Ma B, Shan S, Chen X (2022) Joint feature learning and relation modeling for tracking: A one-stream framework in *Proc. European Conf. Computer Vision*, eds. Avidan S, Brostow G, Cissé M, Farinella GM, Hassner T. pp. 341–357.
54. Kristan M et al. (2019) The seventh visual object tracking vot2019 challenge results in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops*.
55. Kiani Galoogahi H, Fagg A, Huang C, Ramanan D, Lucey S (2017) Need for speed: A benchmark for higher frame rate object tracking in *Int. Conf. Computer Vision*. pp. 1125–1134.
56. Mueller M, Smith N, Ghanem B (2016) A benchmark and simulator for uav tracking in *Proc. European Conf. Computer Vision*. pp. 445–461.
57. Wu Y, Lim J, Yang MH (2013) Online object tracking: A benchmark in *Comp. Vis. Patt. Recognition*. pp. 2411–2418.
58. Tokmakov P, Li J, Gaidon A (2023) Breaking the “object” in video object segmentation in *Comp. Vis. Patt. Recognition*. pp. 22836–22845.
59. Zhou C et al. (2025) Edgetam: On-device track anything model in *Comp. Vis. Patt. Recognition*. pp. 13832–13842.
60. Adamyan A, Čížek T, Straka M, Janouskova K, Schmid M (2025) Sam2rl: Towards reinforcement learning memory control in segment anything model 2. *arXiv preprint arXiv:2507.08548*.
61. Carion N et al. (2025) SAM 3: Segment anything with concepts in *arXiv preprint arXiv:2511.16719*.
62. Missaoui B, Cetintas O, Brasó G, Meinhardt T, Leal-Taixé L (2025) Noougat: Towards unified online and offline multi-object tracking. *arXiv preprint arXiv:2509.02111*.
63. Lv W et al. (2024) Diffmot: A real-time diffusion-based multiple object tracker with non-linear prediction in *Comp. Vis. Patt. Recognition*. pp. 19321–19330.
64. Gao Y, Xu H, Li J, Wang N, Gao X (2024) Multi-scene generalized trajectory global graph solver with composite nodes for multiple object tracking in *Proc. of the AAAI Conf. on Artificial Intelligence*.